

Retail Management System

FINAL YEAR PROJECT PRESENTATION

Presented by :

Sushank Lamsal (79010980)
Manish Karki (79010963)
Sudarshan Magar (79010978)

UNDER THE SUPERVISION OF

Mr. Chakra Rawal





INTRODUCTION

➡ The Retail Management System is a web-based platform designed to support digital retail operations with a focus on intelligence, usability, and system efficiency.

➡ Search products using an image instead of product names.

➡ The defining feature of this system is the integration of AI-based visual search and image recognition technology, enabling users to discover products by uploading images rather than relying on text-based queries.

PROBLEM STATEMENT



Limited Intelligent Features

Traditional systems often lack visual search and personalized shopping capabilities



Fragmented Retail Operations

Inventory, orders, payments and customer management may be handled separately.



Manual Processing

Manual operations can cause data inconsistency, duplicate entries and processing delays.

OBJECTIVES

01

CENTRALIZED PLATFORM

To create a centralized web platform that provide product browsing, cart, ordering, payment and account management.



02

VISUAL SEARCH FUNCTIONALITY

To allow users to upload an image and find visually similar products.



SCOPE



**END-TO-END RETAIL
OPERATIONS**



**AI-DRIVEN VISUAL
DISCOVERY**



**CUSTOMER
ENGAGEMENT**



**SCALABLE
ARCHITECTURE**

LIMITATION



**No real-time analytics
dashboard**



**Server-side rendered
frontend**



**Limited payment
gateways**



**Basic Notification
System**



DEVELOPMENT METHODOLOGY

The project uses the Agile Development Model, which focuses on making progress step by step, getting regular feedback, and easily adjusting to changing needs.

Iteration

01

Foundation

- Django setup
- Database schema
- Authentication
- Product catalog

02

SHOPPING MODULE

- Search & filtering
- Product variations
- Shopping cart

03

ORDER PROCESSING

- Checkout
- eSewa
- COD
- Transaction handling

04

USER ENGAGEMENT

- Reviews
- Ratings
- Dashboard
- Email notifications

05

AI & Testing

- MobileNetV3
- FAISS
- Visual Search
- Testing

REQUIREMENTS ANALYSIS

Non-Functional Requirements

- Usability
- Performance
- Security
- Scalability
- Reliability
- Maintainability

Functional Requirements

- User account management
- Product browsing and search
- Shopping cart
- Order processing
- Payment handling
- Image-based product search
- Email notifications
- Administrative management





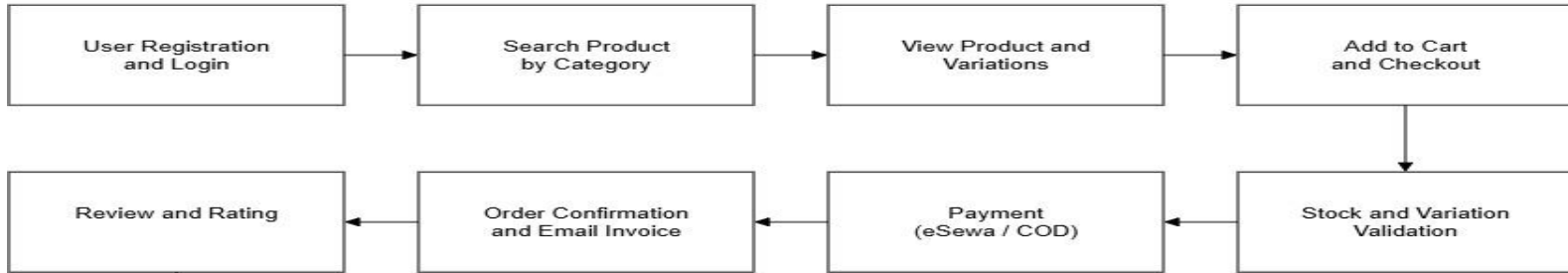
FEASIBILITY STUDY

Feasibility	Summary
Technical	Django, Python, HTML, CSS, JavaScript and supporting ML technologies
Economic	Primarily open-source technologies and standard computing resources
Operational	Browser-based and user-friendly
Legal	Uses legally permissible open-source technologies
Schedule	Agile-based iterative development within the academic timeline



SYSTEM ARCHITECTURE

User Flow



Admin Flow

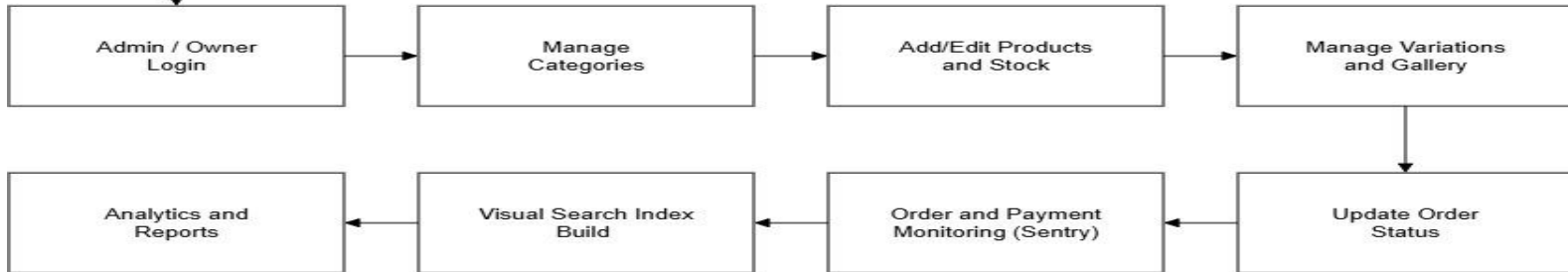


Fig : High-Level System Design of Retail Management System



TECHNOLOGY STACK



FRONTEND

- HTML5 & CSS3
- Bootstrap 5
- Django Template Language
- JavaScript



BACKEND & DB

- Python
- Django 5.2.5
- SQLite3



AI/ML

- PyTorch
- MobileNetV3-Small
- FAISS



DEV & MONITORING

- Git & GitHub
- Sentry
-

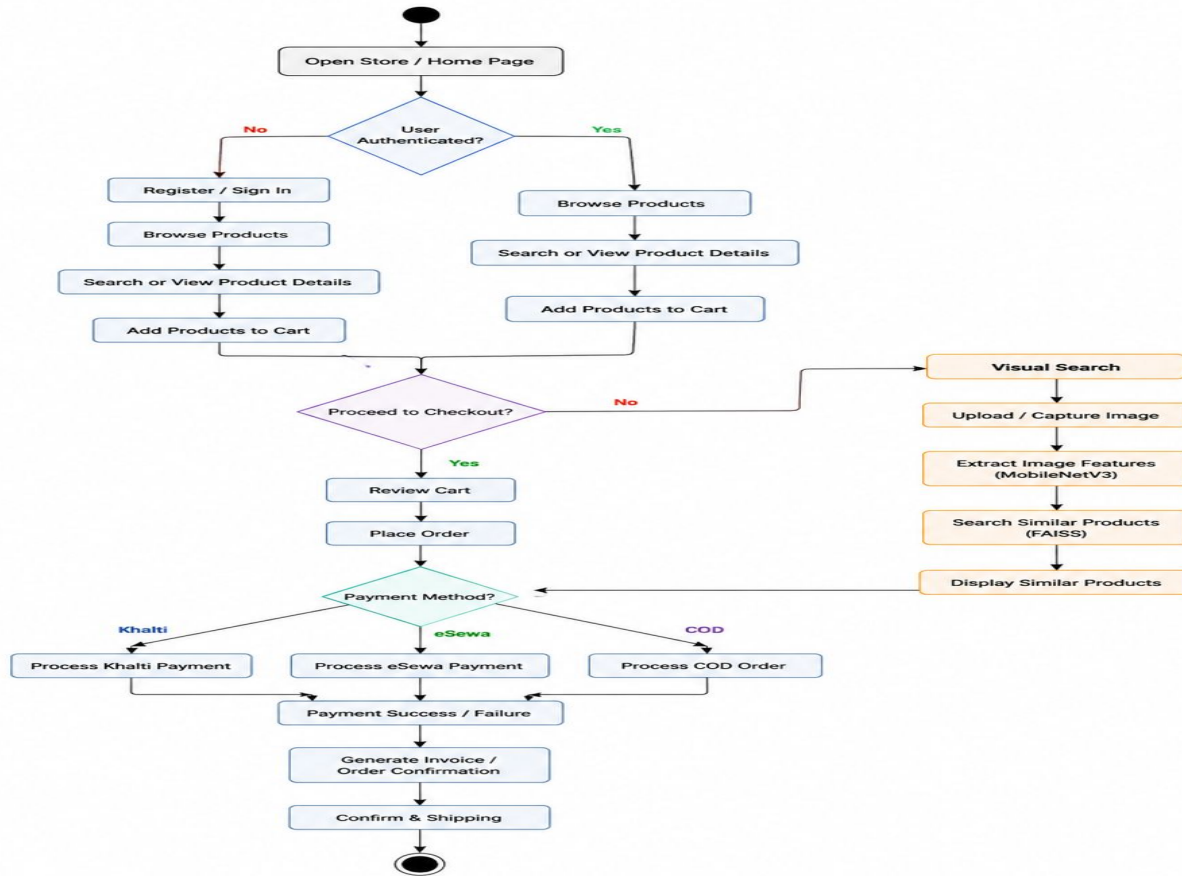


Fig : Activity Diagram of Retail Management System



DESCRIPTION OF ALGORITHM

MobileNetV3-Small

MobileNetV3-Small is a lightweight CNN used as a transfer-learning feature extractor. In this system, the fine-tuned model converts product images into compact 576-dimensional feature embeddings for visual similarity search.

Working

- Preprocesses the input image
- Passes the image through the fine-tuned MobileNetV3-Small.
- Removes the original classification output using nn.Identity.
- Extracts a 576-dimensional feature vector.
- Applies L2 normalization before similarity search.

$$v = f\theta(I)$$

where,

- $f\theta$ = fine-tuned MobileNetV3-Small model
- I = input image
- v = 576-dimensional feature embedding

L2 normalisation

$$\hat{v} = v / \|v\|_2, \text{ with } \|\hat{v}\|_2 = 1$$

FAISS IndexFlatIP

FAISS IndexFlatIP performs exact nearest-neighbor search over the normalized feature embeddings using inner-product similarity to retrieve the most visually similar products.

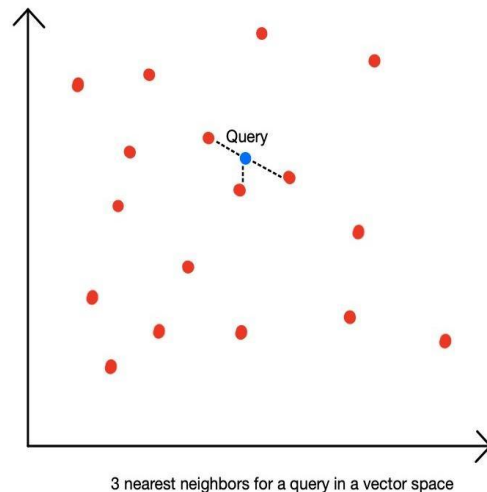
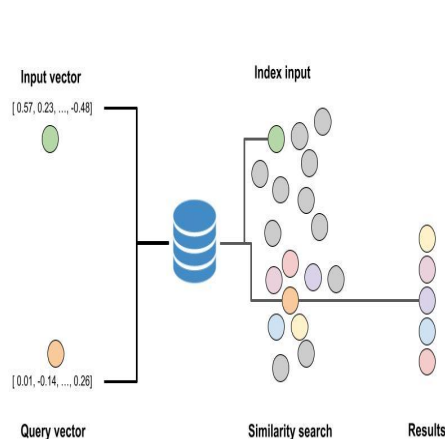
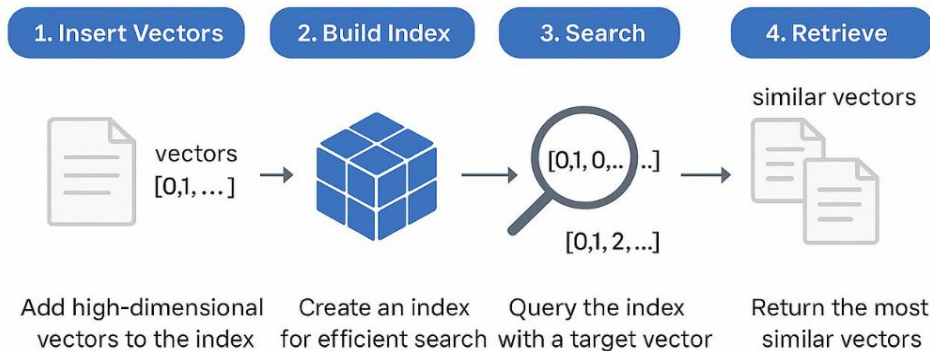
Working

- Product images are converted into 576-D normalized embeddings.
- Embeddings are stored in a FAISS IndexFlatIP.
- A user's image is processed using the same MobileNetV3 pipeline.
- FAISS calculates the inner product between the query and indexed vectors.
- The system ranks products by similarity.
- Top-3 products are returned to the user.

$$s(q, x_i) = q^T x_i = \sum_{j=1}^{576} q_j x_{ij}$$

Top-K retrieval rule

$$R(q) = \text{argtopK}_i s(q, x_i)$$





Thank you

